



Work Integrated Learning – XISD5319

Team name: High Rollers / Client name: SwiftSend
Courier / Refilwe Phore – ST10458818 / Ntembeko Ndayi
– ST10180342 / Project Plan Update

Contents

1. Introduction	2
2. System Requirements	2
2.1 Functional Requirements	2
2.2 Non-Functional Requirements	3
3. Milestone and deliverables	4
4. Work Breakdown Structure (WBS)	4
5. Project Schedule.....	6
Gantt Chart Interpretation	6
DevOps Approach	7
6. Risk Management.....	8
7. Technical Feasibility Issues.....	10
8. Economic Feasibility Issues.....	11
9. Team Members	12
Project Management and Monitoring	15
Conclusion	16
1. Introduction / Problem Domain (System Analysis)	17
2. Functional Requirements Table for SwiftSend Couriers	18
2.2 UML Use Case Diagram.....	18
Appendix A	19
3. Logical System Model Table	20
Appendix B	20
Class Diagrams	21
Appendix C	22
Domain Class Diagram Relationships	22
System Design Document	23
1. Introduction	23
References	28

1. Introduction

SwiftSend Courier is a Johannesburg-based courier company that provides delivery services to customers. The company currently relies on manual processes such as phone calls, paper forms and spreadsheets to manage customer information, delivery requests and driver communication. These processes can result in delays, duplicated records, difficulty tracking deliveries and limited access to management information.

To address these challenges, our team proposes the development of a SwiftSend Courier Management System. The system will consist of a customer website, a driver mobile application and an administrator dashboard. The customer website will allow customers to register, log in, request deliveries, view delivery information and track their parcels. The driver mobile application will allow drivers to view assigned deliveries and update delivery statuses. The administrator dashboard will allow administrators to manage customers, drivers and deliveries and generate reports.

The main purpose of the project is to improve the efficiency of SwiftSend's delivery operations while providing customers with a more convenient way to request and track deliveries. The system will also improve communication between administrators, customers and drivers by providing centralized delivery information and notifications.

The project focuses on developing a functional and user-friendly system that can reduce reliance on manual processes and improve the management of delivery information. Payment integration and live GPS tracking are outside the scope of the current project phase.

2. System Requirements

2.1 Functional Requirements

The SwiftSend Courier Management System should provide the following functionality:

1. Customers must be able to register an account.
2. Customers must be able to log into the system securely.
3. Customers must be able to submit delivery requests.
4. Customers must be able to view their delivery information.
5. Customers must be able to track the status of their deliveries.
6. Drivers must be able to log into the mobile application.
7. Drivers must be able to view deliveries assigned to them.
8. Drivers must be able to update the status of deliveries.
9. Administrators must be able to manage customer information.
10. Administrators must be able to manage driver information.
11. Administrators must be able to manage deliveries.
12. The system must provide notifications relating to delivery status changes.
13. Administrators must be able to access delivery and management reports.

2.2 Non-Functional Requirements

The system should also satisfy the following requirements:

- Usability: The interfaces should be simple and easy to understand.
- Security: User authentication and authorization should protect system information.
- Performance: The system should respond to user requests within a reasonable time.
- Reliability: Delivery information should be stored consistently and accurately.
- Maintainability: The system should be organized so that future improvements can be made.

- Compatibility: The website should work on commonly used web browsers and different screen sizes.
- Data integrity: Customer, driver and delivery information should be stored correctly.

3. Milestone and deliverables

- **Project Planning**
Project plan, responsibilities and schedule
- **Requirements Analysis**
Functional and non-functional requirements
- **UI/UX Design**
Website and mobile application sitemap and wireframes
- **Database Design**
ERD and database structure
- **Website Development**
Functional customer and administrator website
- **Mobile Development**
Functional driver mobile application
- **System Integration**
Website, mobile application, backend and database integration
- **Testing**
Test cases, results and bug fixes
- **Client Review**
Client feedback and required improvements
- **Deployment**
Deployed website and configured system
- **Documentation**
Final documentation and README
- **Final Submission**
Completed and tested project

4. Work Breakdown Structure (WBS)

Task Name	Description	Duration	Predecessor(s)	Team Member	Resources Needed
-----------	-------------	----------	----------------	-------------	------------------

				Responsible	
Project Initiation & Planning	Define project scope. Objectives and requirement	3 days	None	Project Manager, Secretary	Project brief, internet, documentation tools
Requirements Gathering	Collect and analyses system requirement	2 days	Project Initiation	All team Members	Interview notes research materials
Problem Analysis	Identify problem in current system and user needs	2 days	Requirements Gathering	Software Designer, Developer	Research data, case studies
System Design (UI/UX)	Create site maps and wireframes	4 days	Problem Analysis	Software Designer	Figma/Adobe XD design templates
Database Design	Design ERD and database structure	3 days	System Design	Lead Developer	MYSQL, ERD tools

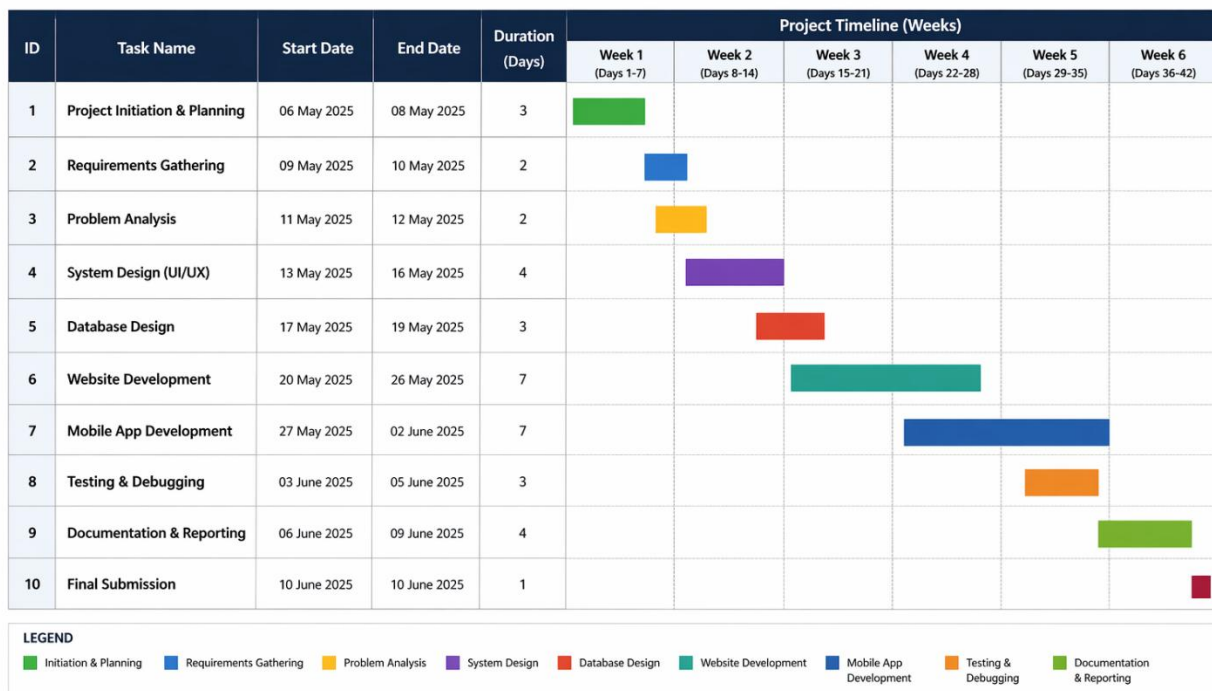
Website Development	Build Functional website with core	7 days	Database design	Lead developer	MYSQL, ERD tools
Mobile App Development	Develop functional mobile application	7 days	Website Developer	All team developers	HTML, CSS JavaScript, PHP, code editor
Testing and Debugging	Test system and fix errors	3 days	Website and App developer	All team developers	Flutter/Android Studio APIs
Documentation & Reporting	Compile final report and	4 days	Testing & Debugging	Secretary/Project Manager	Testing tools, devices bug tracker

	documentati on				
Final Submission	Submit complete system and report	1 day	Documentatio n & Report	Project Manager	MS Word/Google Doc

5. Project Schedule

PROJECT SCHEDULE – GANTT CHART

SwiftSend Couriers Management System



(Meredith, shafer and mantel, 2017)

Gantt Chart Interpretation

- Task List:**
 Shows all project activities outlining all the work required to complete the project (planning, design, development, testing, and submission).
- Start & End Dates:**
 Indicate when each task begins and finishes to ensure proper scheduling.
- Duration:**
 Displays how many days each task will take (e.g. development = 7 days).

- **Timeline (Weeks 1–6):**
Divides the project into a 6-week schedule, helping track progress over time.
- **Task Bars (Colours):**
Coloured bars show when tasks occur and how long they last.
- **Dependencies:**
Tasks follow a sequence (e.g. design must be completed before development starts).
- **Overlapping Tasks:**
Some tasks run at the same time to improve efficiency and reduce delays.
- **Final Milestone:**
Final Submission marks the completion of the project.
- **Legend:**
Explains the meaning of colours used for different project phases.

DevOps Approach

The project will use a basic DevOps approach throughout development. GitHub will be used for version control and to maintain the project source code.

The team will divide the project into smaller tasks and work on them progressively. Code will be tested regularly before integration to reduce the risk of errors affecting the complete system.

The development process will include:

1. Planning.
2. Development.
3. Version control.
4. Testing.
5. Integration.

6. Bug fixing.
7. Deployment.
8. Monitoring and feedback.

6. Risk Management

Risk	Description	Probability	Impact	Proactive Steps to handle the Risk
Team member leaving the group	A member may withdraw from the project before completion	Medium	High	Assign backup role for each task, document all work, and ensure shared access to files (Google Drive/GitHub) so progress is not lost
Lack of team cooperation	Team members do not communicate or work well together	Medium	High	Hold regular meetings, set clear communication channels (WhatsApp/Teams), and define responsibility early using a task allocation sheet
Member does not contribute	Member fails to complete assigned tasks	High	High	Track individual contributions, assign measurable tasks
Lack of technical skills	Team struggles with coding databases, or design tools	Medium	High	Provide training and help from skilled team members
Running behind schedule/Delays	Tasks take longer than planned delaying milestone	High	High	Follow the Gantt chart and priorities important tasks/Schedule control

System errors/bugs	Websites or apps contain technical bugs during development	Medium	Medium	Test regularly during development, not only at the end
Loss of project files	Files may be deleted, corrupted or lost	Low	Very High	Use cloud storage (Google Drive/OneDrive), maintain backups after every session and use version control (e.g. GitHub for code)
Poor requirements understanding	Misinterpretation of project requirements leads to rework	Low	High	Review project brief regularly confirm understand with lecturer/client, and document requirements clearly before development starts
No access to client feedback	Limited or delayed feedback affects decision-making	Medium	Medium	Schedule regular check ins, use written feedback requests and proceed with assumption only when approved by team consensus
Hardware/software failure	Computer or tools used for development fails	Low	High	Use multiple devices, if possible, install required software early, and keep offline backups of all important files and installation.

7. Technical Feasibility Issues

Hardware Resources

Hardware Resource	Availability	Solution (if not available)
Laptop/Computer	Available	N/A
Smartphone (for testing)	Available	Use personal devices
Internet connection	Available	Use home WI-FI or campus network

Software Resources

Software Resource	Availability	Solution (if not available)
Visual Studio Code	Available	Download free from official website
Google Chrome	Available	Install if not already available
Android Studio	Available	Download and install(free)
MYSQL database	Available	Use free community version
Figma/Canva	Available	Use fee online version

All the required hardware and software resources needed for the development of the system are readily available to the team. Most of the tools required are free or open-source, making them easily accessible. In cases where software is not pre-installed at no cost. Therefore, there are no major technical limitations that would prevent the successful development of the system (Laudon and Laudon, 2020)

8. Economic Feasibility Issues

Task Identification	Team Member(s) responsibility	Time allocated per task per team member	Taffir per team member in Rand per hour	Cost per Task (time*tariff)
Project Planning	Refilwe	40	50	2000
Project Planning	Ntembeko	40	50	2000
Requirements Gathering	Refilwe	40	50	2000
Requirements Gathering	Ntembeko	40	50	2000
System Analysis	Refilwe	40	50	2000
System Design	Refilwe	60	50	3000
System Design	Ntembeko	60	50	3000
Site Map & Wireframe	Ntembeko	40	50	2000
Database Design	Refilwe	40	50	2000
Website Development	Refilwe	100	50	5000
Website Development	Ntembeko	100	50	5000
Mobile App Development	Ntembeko	100	50	5000
Testing & Debugging	Refilwe	60	50	3000
Testing & Debugging	Ntembeko	60	50	3000
Documentation	Refilwe	60	50	3000
Documentation	Ntembeko	60	50	3000
Final Submission	Refilwe	20	50	1000
Final Submission	Ntembeko	20	50	1000

Total budget for the project	R49 000
-------------------------------------	----------------

The economic feasibility of the project is determined using an effort-based estimation approach. Each task identified in the work Breakdown structure (WBS) was allocated to specific team members, and the time required to complete each task was estimated in hours. As standard rate of R50 per hour was applied to calculate the cost of each activity

The total project cost was calculated by summing the effort costs of all team members across all tasks. The final estimated cost for the project is R49,000. This indicates that the project is economically feasible, as it mainly requires time and effort rather than significant financial investing

9. Team Members

Team Leader (Project Manager / Developer)

Refilwe Phore ST10458818



- **Responsibilities**

Planning the project, managing timelessness, developing system features, assisting with testing and documentation

- **CV**

Refilwe

refilwephore@gmail.com | 0608147881 | DoB: 10-September-2001

Motivated third-year Information Technology student seeking a Work Integrated Learning (WIL) opportunity to gain practical industry experience. Skilled in web development, database design, and problem-solving with a strong foundation in software development and project management. Eager to contribute technical skills while learning from experienced professionals.

ADDITIONAL CONTACT INFORMATION

- Alternative Contact: 0692051400

EDUCATION

IIE, Rosebank College

01/2024 - present

Diploma in Software Development - Third Year

PROJECTS

Student Event Management Website

- Designed and developed a prototype system for managing student events.
- Created user-friendly interfaces using Human Computer Interaction principles.
- Designed the database and implemented functionality using ASP.NET Core MVC.

Database Management Project

- Created SQL scripts to design and manage relational databases.
- Implemented tables, views, and queries to retrieve and manage data efficiently.

SOFT SKILLS

- Problem-solving
- Teamwork
- Communication
- Time management
- Attention to detail

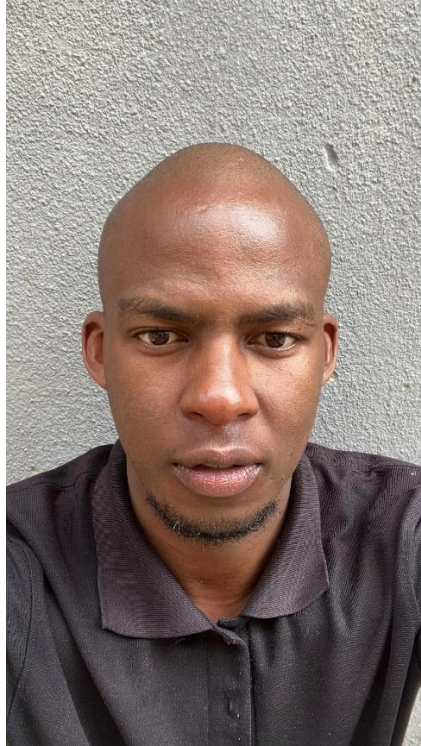
ADDITIONAL INFO

- Available for 6-month WIL placement
- Willing to learn and adapt to new technologies.

KEY SKILLS

- Programming / Development
- HTML
- CSS
- JavaScript
- ASP.NET Core MVC
- Database

Lead Developer
Ntembeko Ndayi ST10180342



- **Responsibilities**
Designing the user interface developing system components, assisting with testing and documentation
- **CV**

CV

Ntembeko Ndayi

Personal Details

Name

Ntembeko Ndayi

Email address

ntembekondayi@gmail.com

Phone number

0731234567

Address

123 Capricorn Avenue, 2190 Posetianville, Johannesburg

Summary

Software Developer with a strong foundation in web and mobile application development. Passionate about designing user-friendly interfaces and building efficient system components. Committed to delivering high quality solutions and continuously improving technical and problem solving skills.

Education

Feb 2023 - Nov 2025

Diploma In Software Development
Rosebank College, Braamfontein, Johannesburg

Technical Skills

Programming: HTML, CSS, JavaScript, C#

Database: MySQL

Tools: Visual Studio Code, Android Studio, ProjectLibre, Github

Key Strengths

- Problem-solving
- Attention to detail
- Team collaboration
- Time management

References

References available upon request.

Project Management and Monitoring

The progress of the project will be monitored by comparing completed activities with the project schedule.

The team will:

- Track completed tasks.
- Monitor deadlines.
- Communicate regularly.
- Test completed functionality.
- Record and resolve bugs.
- Review client feedback.
- Update the project documentation.
- Maintain the GitHub repository.

This will help our team identify delays or problems early and make the necessary adjustments.

Conclusion

The SwiftSend Courier Management System is intended to improve the way SwiftSend manages customers, drivers and deliveries. The current manual processes create challenges such as delays, duplicated information and limited delivery tracking.

The proposed system will provide a customer website, driver mobile application and administrator dashboard. These components will work together to support delivery booking, tracking, driver status updates, notifications and management reporting.

The updated project plan provides a structured approach for completing the project through planning, requirements analysis, design, development, integration, testing, deployment and documentation. By following the planned schedule and using regular testing and version control, the team aims to produce a functional and user-friendly system that addresses the client's identified needs.

1. Introduction / Problem Domain (System Analysis)

SwiftSend Couriers is a Johannesburg-based delivery company that currently operates without an integrated digital system. The company manages orders, customers, and deliveries using manual methods such as phone calls, paper forms, and spreadsheets (Laudon & Laudon, 2020).

A study of the organisation identified the following specific problems:

- Customers cannot book deliveries online or track parcels in real time
- Customer records are stored manually in spreadsheets, leading to errors and duplicate entries
- Drivers receive delivery instructions by phone, resulting in delays and miscommunication
- Management has no access to reports on delivery performance or driver efficiency (Meredith et al., 2017)

To address these problems, this project proposes a Courier Management System that will provide:

- Online delivery booking for customers
- Real-time parcel tracking
- Automated email/SMS notifications
- A driver mobile app for status updates
- An admin dashboard for managing records and generating reports

Scope: The system includes customer registration, delivery requests, tracking, notifications, driver assignment, and basic reporting. Payment integration and live GPS tracking are excluded from this phase.

Benefits if successful: Improved customer satisfaction, operational efficiency, and competitive advantage.

Risks if the project fails: Continued inefficiencies, customer loss, and inability to compete with digital-first courier companies.

2. Functional Requirements Table for SwiftSend Couriers

Participant (Active actor)	Function of the system	Participant (Passive actor)
Customer	Register a new account	System
Customer	Logs into the system	System
Customer	Places a delivery request	System
Customer	Tracks a parcel in real time	System
Customer	Receives delivery notification	System
Driver	Logs into mobile app	System
Driver	Views assigned deliveries	System
Driver	Update delivery status (pick up, in transit, delivered)	Customer, System
Administrator	Logs into admin dashboard	System
Administrator	Manages customer records	System
Administrator	Manages driver records	System
Administrator	Assign deliveries to the driver	Driver
Administrator	Generates delivery reports	System
System	Send automated email/SMS notification	Customer, Driver

(Dennis et al., 2015)

2.2 UML Use Case Diagram

The UML Use Case Diagram for the SwiftSend Courier Management System includes the following components:

Actors:

Customer – uses the website to register, log in, place delivery requests, and track parcels

Driver – uses the mobile app to view assigned deliveries and update delivery status

Administrator – uses the admin dashboard to manage customers, drivers, and deliveries, and generate reports

Notification System – an external system that sends automated email and SMS notifications

Subsystems:

Customer Subsystem

Driver Subsystem

Admin Subsystem

Use Cases by Subsystem:

Subsystem	Use Case
Customer Subsystem	Register Account, Login, Place Delivery Request, Track Parcel, Receive Notification
Driver Subsystem	Login, View Assigned Deliveries, Update Delivery Status
Admin Subsystem	Login, Manage Customers, Manage Drivers, Assign Deliveries, Generate Reports

Relationships:

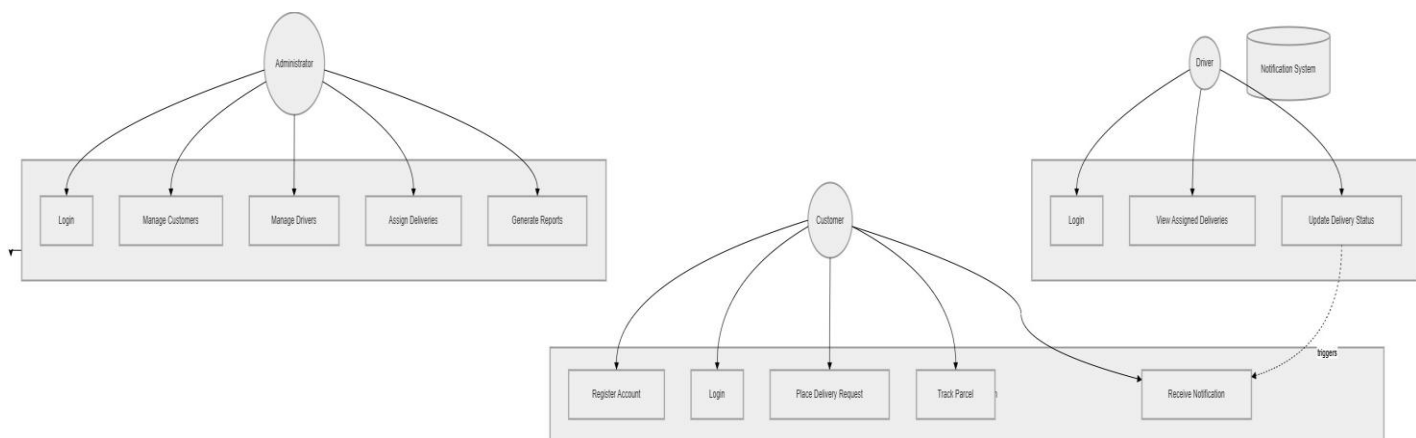
The Customer actor is associated with all Customer Subsystem use cases

The Driver actor is associated with all Driver Subsystem use cases

The Administrator actor is associated with all Admin Subsystem use cases

When the driver updates the delivery status, it triggers the system to send a notification to the customer

Appendix A



3. Logical System Model Table

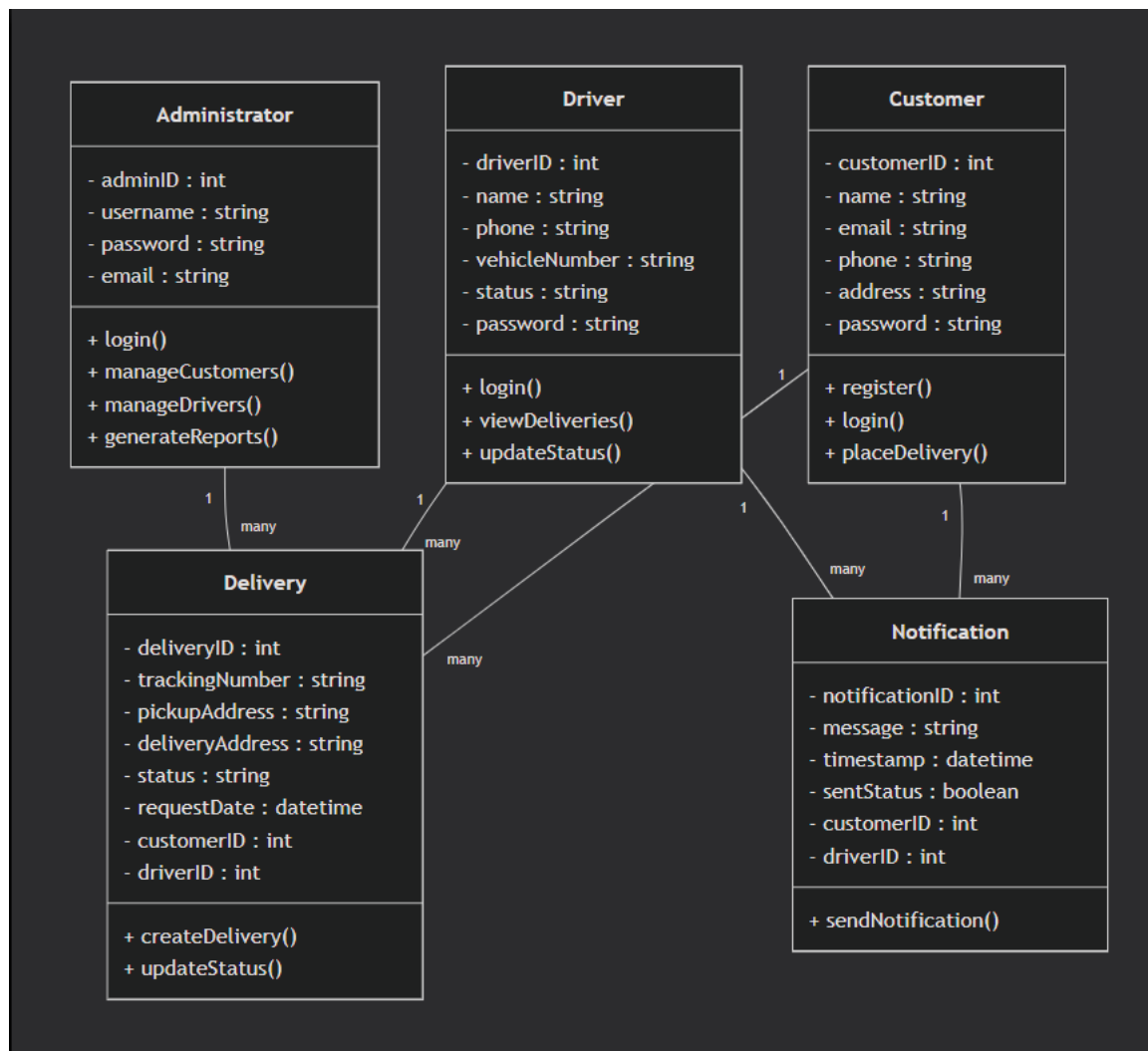
Appendix B

GUI	System Process (Method)	Entity Relationship (Table)	
Input	Output		
Enter name, email, phone, address	Registration confirmation	Registering new customer	Customer table
Enter email and password	Login success message	Authenticate user	Customer table/ Driver table/ Admin table
Enter pickup address, delivery address, parcel details	Booking confirmation and tracking number	Create delivery request	Delivery table
Enter tracking number	Parcel status and location	Retrieve tracking information	Delivery table
No input	Delivery status notification	Send automated notification	Delivery table, Notification table
Driver enters login credentials	List of assigned deliveries	Fetch assigned deliveries	Delivery table, Driver table
Driver selects delivery ID and update status (Picked up/in transit/delivered)	Status update confirmation	Update delivery status	Delivery table
Administrator enters login credentials	Admin dashboard with orders	Load dashboard data	Delivery table, Customer table, Driver table
Administrator enters driver's name, vehicle, phone	Driver added confirmation	Add new driver	Driver table
Administrator selects date range	Delivery report on screen or printer	Generate delivery report	Delivery table

Class Diagrams

Name of entity (UML Class)	Property of entity (UML Class)	Related to
Customer	- customerID(int) - name (string: 50) - email (string: 100) - phone (string: 15) - address (string: 200) - password (string: 255)	Delivery, Notification
Administrator	- adminID (int) - username (string: 30) - password (string: 255) - email (string: 100)	Delivery, Customer, Driver
Driver	- driverID (int) - name (string: 50) - phone (string: 15) - vehicleNumber (string: 20) - status (string: 20) - password (string: 255)	Delivery
Delivery	- deliveryID (int) - trackingNumber (string: 20) - pickupAddress (string: 200) - deliveryAddress (string: 200) - status (string: 20) - requestDate (datetime) - customerID (int) - driverID (int)	Customer, Driver
Notification	- notificationID (int) - message (string: 500) - timestamp (datetime) - sentStatus (boolean) - customerID (int) - driverID (int)	Customer, Driver

Appendix C



Domain Class Diagram Relationships

The domain class diagram includes the following relationships:

- **Association:** One Customer can place many Deliveries. One Driver can be assigned many Deliveries. One Administrator manages many Deliveries.
- **Association:** One Customer can receive many Notifications. One Driver can receive many Notifications.
- **Aggregation:** The Administrator manages Customer, Driver, and Delivery records, but these entities can exist independently.
- **Generalization:** No generalization relationships are present in this phase.

System Design Document

1. Introduction

SwiftSend Couriers is a delivery company based in Johannesburg that currently uses manual methods such as phone calls, paper forms, and spreadsheets to manage deliveries. These outdated processes often result in delays, incorrect records, poor communication, and customer dissatisfaction.

The proposed SwiftSend Courier Management System (CMS) aims to modernize operations through a digital platform that allows customers to book deliveries online, track parcels in real time, and receive automatic notifications. Drivers will use a mobile application to update delivery progress while administrators manage operations through a web-based dashboard.

The system follows a three-tier client server architecture consisting of the Presentation Layer, Application Layer, and Data Layer. This approach improves scalability, maintainability, and system performance.

2. Logical Architectural Design

The system uses a client server architecture model suitable for different user groups.

High Level Components:

- Frontend: Responsive web application for customers and administrators, and a mobile application for drivers.
- Database: Relational database for storing system information.
- External Services: Notification service for SMS and email communication.

Lower Level Design:

- JSON is used for communication between the client and server.
- JWT authentication tokens are used for secure session management.
- The architecture supports parcel booking, status updates, report generation, and delivery assignment.

3. User Interaction Design – Input Interactions

- Customer Web Portal:
 - Registration form with customer details.
 - Secure login functionality.
 - Delivery booking form for pickup and delivery information.
 - Parcel tracking page using tracking numbers.

Driver Mobile App:

- Driver login screen.
- Dashboard displaying assigned deliveries.
- Delivery status update page with proof of delivery options.

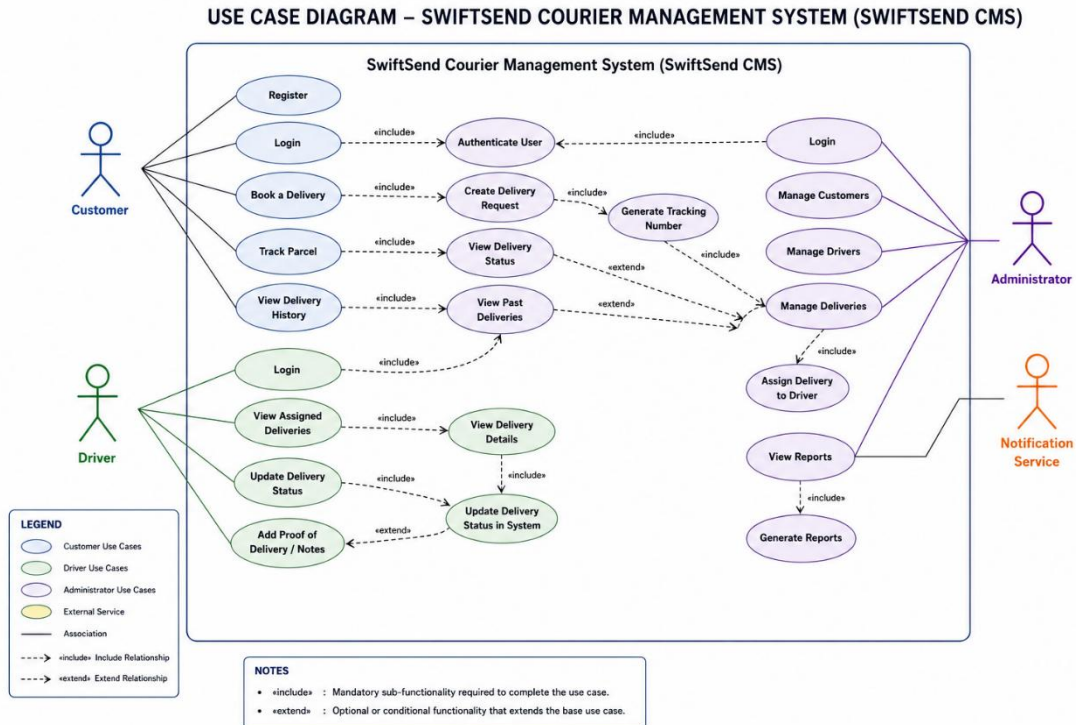
Admin Dashboard:

- Customer and driver management pages.
- Delivery assignment section.
- Reporting interface with date filters.

All forms include validation, helpful messages, and confirmation notifications.

4. User Interaction Design – Request Interactions

- Customers can place delivery requests online.
- The system automatically generates unique tracking numbers.
- Drivers update parcel status in real time.
- Customers receive notifications when delivery status changes.
- Administrators can assign pending deliveries to available drivers.
- Reports are generated based on selected date ranges and criteria.



5. Database Design – Tables

Customers Table

- customerID (PK)
- name
- email
- phone
- address
- password
- createdAt

Drivers Table

- driverID (PK)
- name
- phone
- vehicleNumber
- status
- password

Deliveries Table

- deliveryID (PK)
- trackingNumber (Unique)
- pickupAddress

Delivery Address

- status
- requestDate
- updatedAt
- customerID (FK)
- driverID (FK)

Notifications Table

- notificationID (PK)
- message
- timestamp
- deliveryID (FK)
- customerID (FK)
- driverID (FK)

Administrators Table

- adminID (PK)
- username
- password
- role

6. Database Design – ERD

The Entity Relationship Diagram (ERD) illustrates the relationships between the main system entities:

- One Customer can place many Deliveries.
- One Driver can handle many Deliveries.
- One Delivery can generate multiple Notifications.
- Customers and Drivers can receive Notifications.

Entity Relationship Diagram (ERD)

SWIFTSEND COURIERS – ERD

Courier Management System Database Design

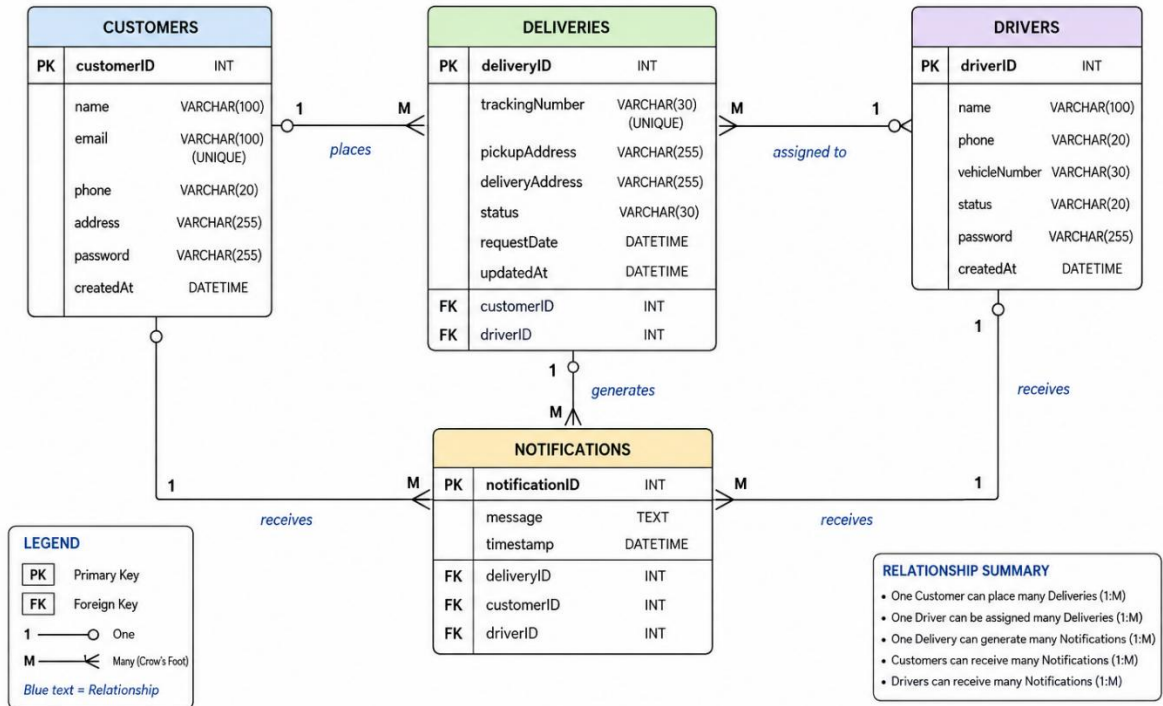


Figure 1: SwiftSend Courier Management System ERD

The database structure is fully normalized and supports all required business operations.

7. Report Design

- Delivery Performance Report.
 - Displays delivery details, drivers, customers, and parcel status.
 - Includes statistics such as total deliveries and successful deliveries.

Driver Efficiency Report:

- Displays completed deliveries per driver.
- Shows average delivery times and driver activity.

Reports can be filtered by date and exported to PDF formats.

8. Conclusion

The SwiftSend Courier Management System provides a reliable solution for improving delivery operations. The proposed design modernises manual processes, improves efficiency, enhances customer experience, and supports future expansion.

References

Dennis, A., Wixom, B.H. and Tegarden, D.P., 2015. *Systems analysis and design: An object-oriented approach with UML*. 5th ed. Hoboken: Wiley.

Kerzner, H., 2017. *Project management: A systems approach to planning, scheduling, and controlling*. 12th ed. Hoboken: Wiley.

Laudon, K.C. and Laudon, J.P., 2020. *Management information systems: Managing the digital firm*. 16th ed. Harlow: Pearson.

Meredith, J.R., Shafer, S.M. and Mantel, S.J., 2017. *Project management: A managerial approach*. 9